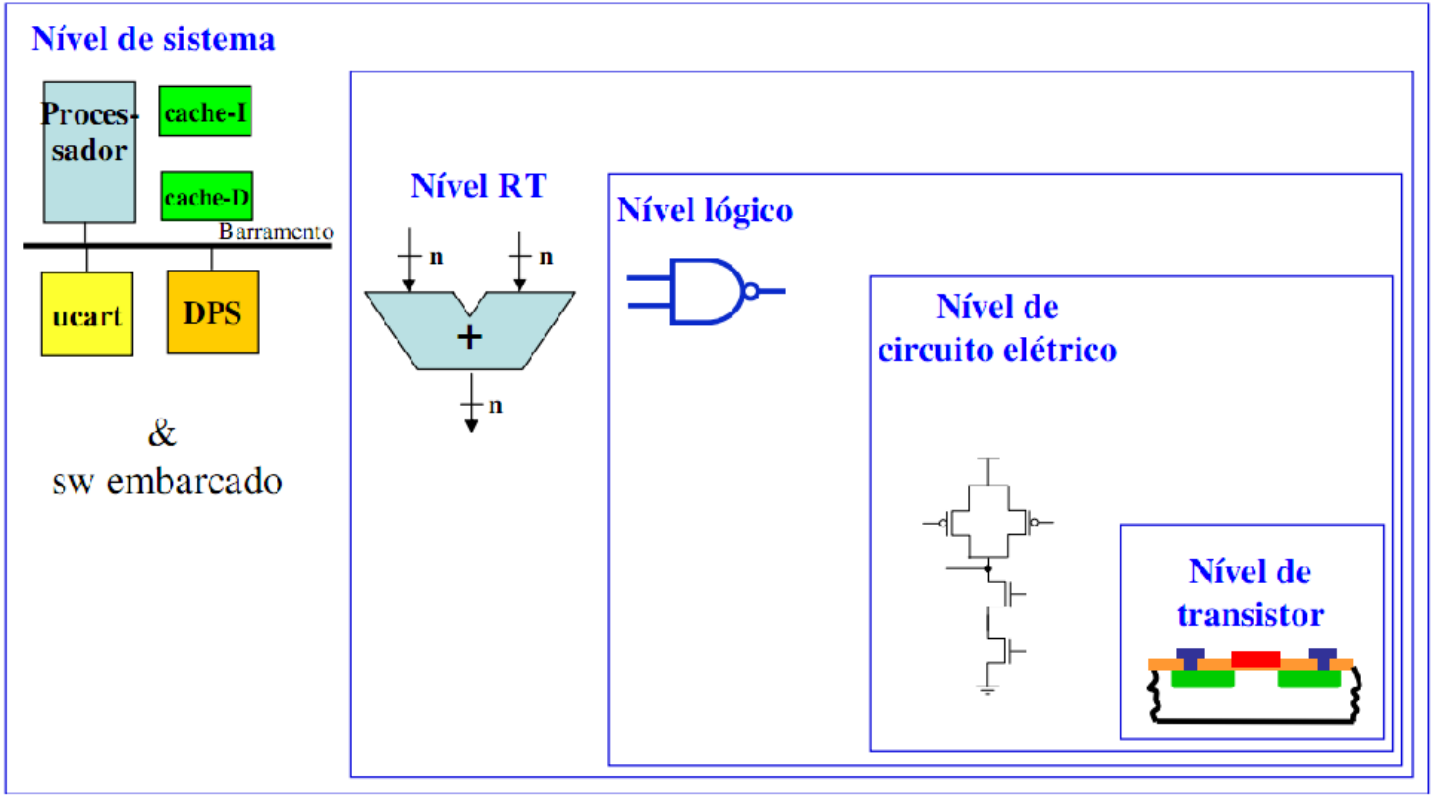
Universidade Federal da Fronteira Sul
Curso de Ciência da Computação
Campus Chapecó

Verilog HDL

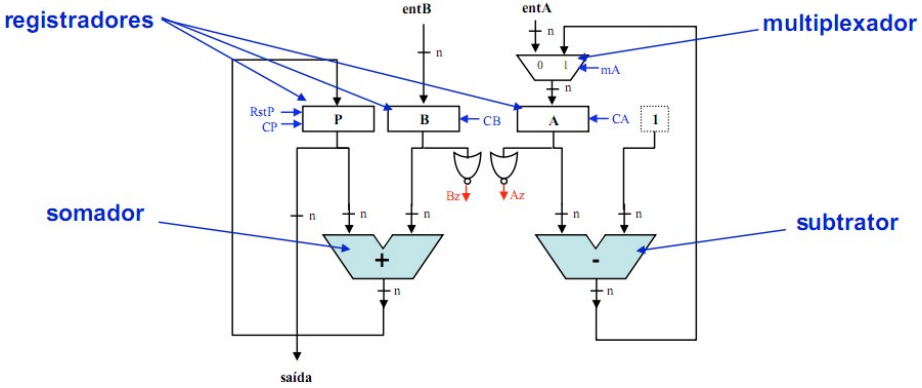
Prof. Luciano L. Caimi
lcaimi@uffs.edu.br

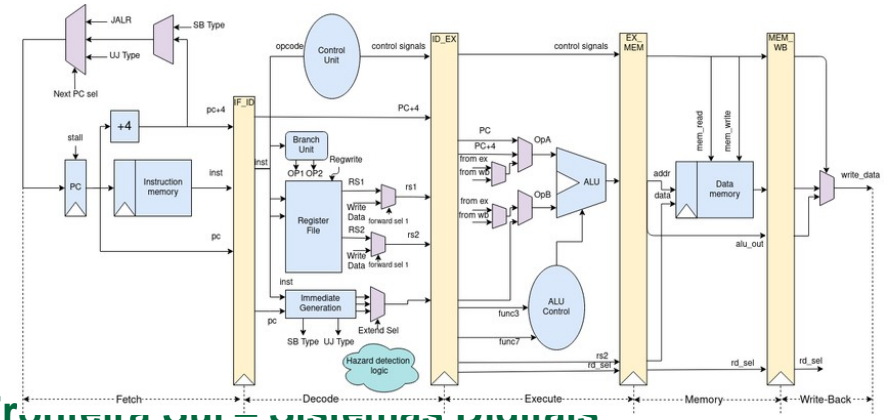
Introdução


Nível de sistema
Proces-
sador
cache-I
cache-D
Barramento
uart
DPS
&
sw embarcado
Nível RT
n
n
+
n
Nível lógico
Nível de
circuito elétrico
Nível de
transistor

Nível de Transferência entre Registradores (RT)

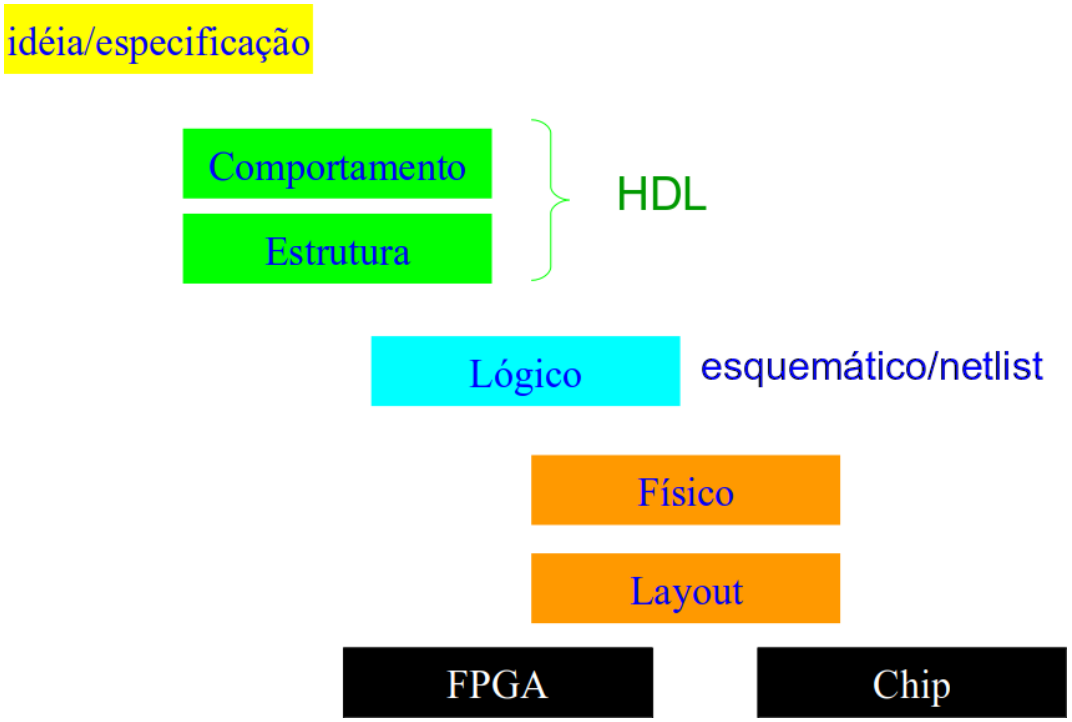
- Unidades funcionais (somadores, subtratores, somadores/subtratores, multiplicadores etc)
- Rede de interconexão (fios, multiplexadores, decodificadores, barramentos, buffers tri-state)
- Registradores e blocos de memória RAM, ROM etc



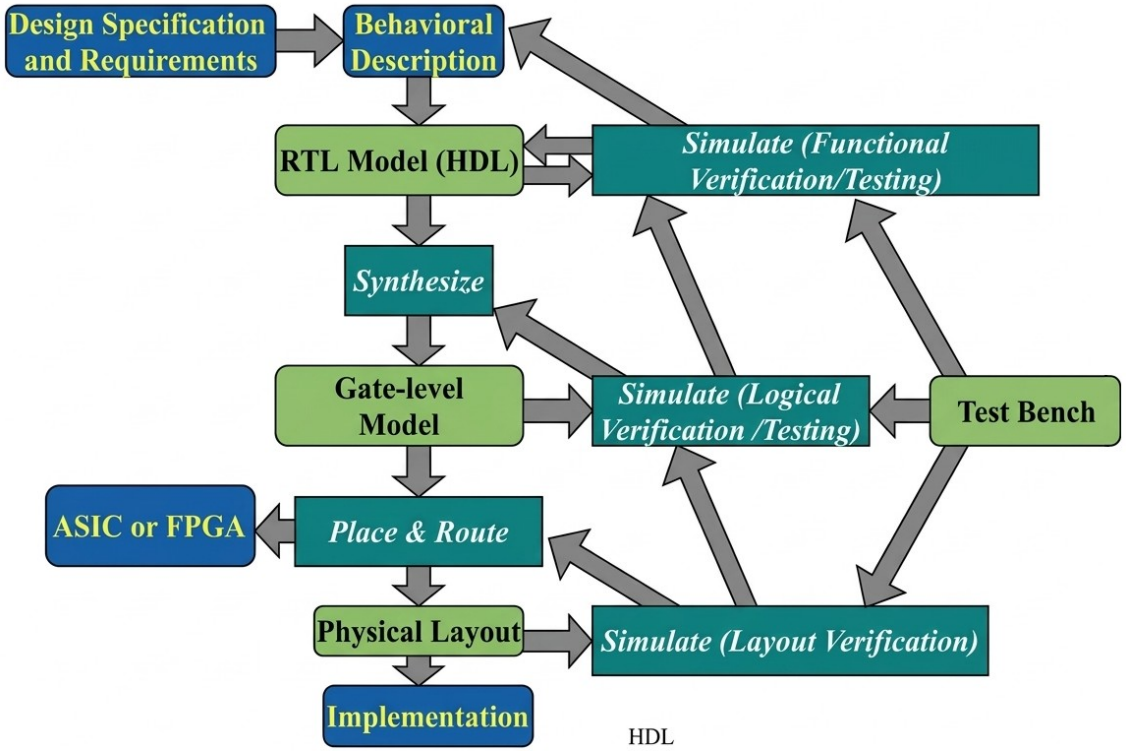


Introdução

- Fluxo de Projeto (simplificado)



Introdução


Design Specification and Requirements
Behavioral Description
RTL Model (HDL)
Simulate (Functional Verification/Testing)
Synthesize
Gate-level Model
Simulate (Logical Verification /Testing)
Test Bench
ASIC or FPGA
Place & Route
Physical Layout
Simulate (Layout Verification)
Implementation
HDL

Verilog: histórico e contexto

HDL (Hardware Description Language)

São linguagens utilizadas para descrever o comportamento e a estrutura de circuitos digitais. Permitem simular, verificar e sintetizar hardware antes da fabricação, sendo essenciais no desenvolvimento de FPGAs e ASICs

1984
Verilog criado pela Gateway Design Automation

1995
Verilog torna-se padrão IEEE 1364-95 (atualizado em 2001)

2002
SystemVerilog lançado como extensão

2005
SystemVerilog vira padrão IEEE 1800 (unificado)

O que é Verilog?

- Criada em 1984, padronizada em 1995 (IEEE 1364)
  - Inventada por Philip Moorby
- Focada em descrição estrutural e comportamental de hardware
- Tipos de dados básicos:

  wire, reg, integer
- Lógica de 4 valores:

  0, 1, X (desconhecido), Z (alta impedância)
- Suporte nativo a simulação e síntese RTL
- Amplamente usada em projetos e indústria

O que é System Verilog?

- Extensão do Verilog (IEEE 1800, desde 2005)
- Combina HDL + HVL (**Hardware Verification Language**)
- Suporte a programação orientada a objetos (OOP)
- Novos tipos: logic, bit, byte, int, enum, struct
- Interfaces, clocking blocks, assertions (SVA)
- Base do framework UVM para **verificação avançada**

Comparativo

Característica	Verilog	SystemVerilog
Padrão IEEE	**1364**	**1800**
Tipos de dados avançados	✗	✓
Orientação a objetos	✗	✓
Randomização constrained	✗	✓
Assertions (SVA)	✗	✓
Interfaces	✗	✓
Síntese de hardware	✓	✓ **(subconjunto)**
Compatibilidade retroativa	—	✓ **(inclui Verilog)**

Quando usar cada um?

Use Verilog quando...

- Trabalhar em projetos legados que já usam Verilog
- A ferramenta de síntese não suporta SystemVerilog
- **Aprender HDL do zero (conceitos mais simples)**
- Fazer pequenos módulos RTL sem necessidade de verificação avançada

Use SystemVerilog quando...

- Criar ambientes de verificação completos e robustos
- Trabalhar com UVM em projetos profissionais
- Usar assertions para verificar protocolos complexos
- Fazer projetos acadêmicos/profissionais modernos

Comparativo

Característica	Verilog (IEEE 1364)	VHDL (IEEE 1076)	SystemVerilog (IEEE 1800)
Origem	Criado pela Gateway Design Automation (1984) como linguagem comercial.	Criado pelo DoD (EUA) nos anos 80 para padronização militar.	Evolução do Verilog com doações da Co-Design Automation (Superlog).
Sintaxe	Baseada na linguagem **C** (concisa e direta).	Baseada em **Ada/Pascal** (verbosa e estruturada).	Baseada em **C/C++** (estende o Verilog com conceitos de POO).
Tipagem	**Fraca**: Permite atribuições implícitas entre tipos diferentes.	**Forte**: Exige conversões explícitas de tipos (mais rígida).	**Híbrida**: Mantém a flexibilidade do Verilog, mas com tipos complexos.
Abstração	Baixa a Média (focada em portas e RTL).	Média a Alta (excelente para sistemas complexos).	**Altíssima** (suporta transações e modelos de sistema).
Filosofia	"Faça rápido e simples" (estilo programador).	"Documente e garanta a estrutura" (estilo engenheiro).	"Descreva o hardware e verifique-o no mesmo lugar".
Poder de Verificação	**Baixo**: Limitado a testbenches manuais e simples.	**Médio**: Robusto para lógica, mas limitado para verificações modernas.	**Máximo**: Suporta POO, Assertions (SVA) e Randomização.
Uso Principal	Síntese de circuitos integrados e ASICs simples.	FPGAs, Defesa, Aeroespacial e Projetos Europeus.	**Padrão da Indústria** para Verificação e Design de SoCs.

Onde estudar e praticar Verilog

Site	Link	Nível	Tipo de Conteúdo	Prática Online	Diferencial	Indicado para
Nandland	https://nandland.com/learn-verilog/	Iniciante	Tutoriais + exemplos	❌	Explicações simples e didáticas	Primeiro contato com Verilog
ChipVerify	https://www.chipverify.com/verilog	Básico → Avançado	Referência + exemplos	✅	Simulação no navegador + muitos exemplos	Aprendizado completo e progressivo
HDLBits	https://hdlbits.01xz.net	Básico → Intermediário	Exercícios	✅	Correção automática tipo "judge"	Fixação prática intensiva
ASIC-World	https://www.asic-world.com	Intermediário	Tutoriais teóricos	❌	Abordagem mais formal/acadêmica	Base conceitual sólida
Learn VLSI	https://www.learn-vlsi.com	Intermediário → Avançado	Tutoriais + contexto	❌	Integra Verilog com fluxo de projeto real	Visão de indústria
Hackr.io	https://hackr.io/tutorials/learn-verilog	Todos	Curadoria de cursos	❌	Junta várias fontes externas	Encontrar materiais diversos
Doulos	https://www.doulos.com	Avançado	Treinamento profissional	❌	Conteúdo usado na indústria	Aperfeiçoamento profissional

Simulação online: www.edaplayground.com

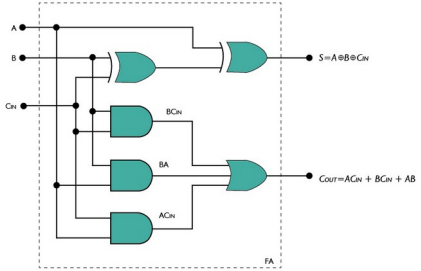
Para lembrar

Linguagem de Descrição de Hardware

NÃO É

Linguagem de Programação

- HDL é concorrente: o processamento ocorre de forma paralela



verilog

```verilog
module somador_completo (
    input a,
    input b,
    input cin,
    output s,
    output cout
);
    // Lógica da soma
    assign s = a ^ b ^ cin;
    // Lógica do transporte de saída (Carry Out)
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

Unidade básica de projeto em Verilog: **Module**

Especifica as entradas e saídas do módulo de hardware e o seu comportamento

```
module [nome] ( [lista_de_portas] );

    [lista_de_portas]
    [lista_de_portas]

    [declaração de outros sinais]

    [instâncias de outros módulos]

    [código do comportamento desse módulo]
endmodule
```

Exemplo: Meio Somador

```
module meio_somador(
  input a, b,
  output soma, carry);
  assign soma = a ^ b; // ^ ou exclusivo
  assign carry = a & b; // & and
endmodule
```

Verilog

Sistema de Lógica de 4 Valores:

0	Nível lógico baixo	1	Nível lógico alto	X	Desconhecido / indefinido	Z	Alta impedância

NETS (Fios)	
wire	Fio simples — conexão padrão
wand	Wired-AND — lógica coletor aberto
wor	Wired-OR — lógica emissão aberta
tri	Tri-state — controle de barramento
supply0	Ligação direta ao GND (0)
supply1	Ligação direta ao VCC (1)

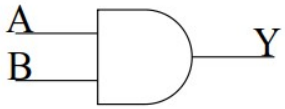
VARIÁVEIS	
reg	Armazena valor lógico (4 estados)
integer	Inteiro 32-bit (síntese limitada)
real	Ponto flutuante (só simulação)
time	Tipo de tempo (64-bit unsigned)
realtime	Tempo em ponto flutuante

EXEMPLOS

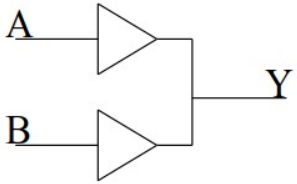
```
// Declaração de nets
wire a, b, c;
wire [7:0] bus;

// Variáveis
reg q;
reg [3:0] cnt;
integer i;

// Vetores e arrays
reg [7:0] mem[0:255];

// Constantes
parameter W = 8;
localparam MAX = 255;
```

Verilog

EXEMPLOS




```
wire Y;  // declaration
assign Y = A & B;
```

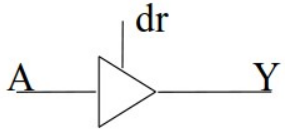



```
wand Y;  // declaration
assign Y = A;
assign Y = B;
```

Y \ A	0	1
B = 0	0	0
B = 1	0	1

```
wor Y;  // declaration
assign Y = A;
assign Y = B;
```

Y \ A	0	1
B = 0	0	1
B = 1	1	1




```
tri Y;  // declaration
assign Y = (dr) ? A : z;
```

Verilog

Palavras reservadas:

Estrutura de Módulo

`module` `endmodule` `port`
`inout` `input` `output`
`parameter` `localparam`

Tipos de Dados

`wire` `reg` `integer` `real`
`time` `tri` `supply0` `supply1`

Controle de Fluxo

`if` `else` `case` `casex`
`casez` `endcase` `for` `while`
`repeat` `forever`

Blocos Procedurais

`always` `initial` `begin` `end`
`fork` `join` `task` `function`

Eventos e Temporização

`posedge` `negedge` `#` `@`
`wait` `disable`

Primitivas

`and` `or` `not` `nand` `nor`
`xor` `xnor` `buf` `bufif0`
`bufif1`

Verilog

Operadores:

Aritméticos		Lógicos / Bit a Bit		Relacionais		Shift / Concat	
+	Soma	&	AND	==	Igual	<<	Shift L
-	Subtração	|	OR	!=	Diferente	>>	Shift R
*	Multiplicação	^	XOR	<	Menor	<<<	Arith L
/	Divisão	~	NOT	>	Maior	>>>	Arith R
%	Módulo	~&	NAND	<=	Menor igual	{,}	Concat
**	Potência	~|	NOR	>=	Maior igual	{{}}	Replicação

Exemplo de uso de operadores

```
assign soma = a + b;    // aritmético
assign flag = (x == y) & (z != 0);  // relacional + lógico
assign word = {byte_h, byte_l};   // concatenação
assign rep = {4{bit_in}};  // replicação
```

Verilog

Números:

- Por default números são decimais: seg = 12; // seg = 7’b0001100;
- Para outro formato (hexa, binário, octal):

[nro de bits]’[base][valor]

↓

Binário = b
Octal = o
Hexadecimal = h
Decimal = d

- Exemplos:

val = 3’b010; // 2 em binário

tam = 6’o76; // 62 em decimal

bar = 8’h70; // 112 em decimal

Verilog

Vetores e arrays:

- Ao declarar um **vetor** *wire* ou *reg* usamos a seguinte sintaxe:

<tipo> <MSB:LSB> <nome>;

ou <tipo> <LSB:MSB> <nome>;

- Exemplos:

```
wire [3:0] bar;      // 'fio' com 4 bits
wire [0:6] other;    // 'fio' com 6 bits – cuidado com a ordem
reg [7:0] out;       // reg com 8 bits

wire x1;                 // x1 não é vetor
assign x1 = bar[3]; // bit 3 do vetor é atribuido a x1
```

Verilog

Vetores e arrays:

- Ao declarar um **array** *wire* ou *reg* usamos a seguinte sintaxe:

  <tipo> <tamanho> <nome><elementos>;

  <tipo> <MSB:LSB> <nome><MSB:LSB>;

- Exemplos:

```
reg y1 [11:0];  // um array de 12 posições com um bit
wire [0:7] y2 [3:0]  // um array de 4 posições com 8-bit cada
reg [7:0] y3 [0:1][0:3]; // um array 2D com 2 linhas e 4 colunas, 8 bits cada
```

Abstrações

HDLs em geral e Verilog possuem 3 tipos de abstração (modelagem) para implementação de circuitos:

Estrutural:	nível de registradores e portas lógicas	and, nand, or, nor, xor, xnor, not
Fluxo de Dados:	descreve funcionalidade (como o dado flui) e não portas lógicas	assign e operadores: |, &, ^, ~, ~&, ~|
Comportamental:	alto nível onde a lógica do comportamento é especificada (blocos procedurais)	always, @, initial, if-else, for

Verilog

ESTRUTURAL
Instanciação de portas

```
module ms_est(
  input a, b,
  output s, c
);
  xor X1(s,a,b);
  and A1(c,a,b);
endmodule
```

FLUXO DE DADOS
Assign contínuo

```
module ms_fd(
  input a, b,
  output s, c
);
  assign s = a ^ b;
  assign c = a & b;
endmodule
```

COMPORTAMENTAL
Bloco always

```
module ms_beh(
  input a, b,
  output reg s, c
);
  always @(*) begin
    s = a ^ b;
    c = a & b;
  end
endmodule
```

Comparativo

Aspecto	Estrutural	Fluxo de Dados	Comportamental
Abstração	Baixa	Média	Alta
Próximo de	Hardware físico	Lógica booleana	Software/Algoritmo
Lógica sequencial	Instanciar flip-flops	Limitado	always @(posedge clk)
Síntese	Direta	Boa	Depende do estilo

Verilog

ESTRUTURAL
Instanciação de portas

```
module mux4x1_estrutural (
    input [3:0] i,   // 4 entradas dados (i0, i1, i2, i3)
    input [1:0] sel, // 2 linhas de seleo (s1, s0)
    output y         // 1 sada
);
    // Sinais internos para os inversores e portas AND
    wire s1n, s0n;
    wire a0, a1, a2, a3;

    // Inversores para as linhas de selecao (NOT)
    not n1 (s1n, sel[1]);
    not n0 (s0n, sel[0]);

    // Portas AND de 3 entradas para selecionar os dados
    and g0 (a0, i[0], s1n, s0n); // 00
    and g1 (a1, i[1], s1n, sel[0]); // 01
    and g2 (a2, i[2], sel[1], s0n); // 10
    and g3 (a3, i[3], sel[1], sel[0]); // 11

    // Porta OR para combinar os resultados
    or final_mux (y, a0, a1, a2, a3);
endmodule
```

Verilog

ESTRUTURAL
Instanciação de portas

Instanciação: Para usar um módulo dentro de outro cria-se uma instância com nome único. Cada porta é conectada a um sinal do módulo pai.

Por Posição (positional)

```
somador_completo u0(
  a_in,  // 1ª porta → a
  b_in,  // 2ª porta → b
  c_in,  // 3ª porta → soma
  s_out, // 4ª porta → carry
  c_out  // 5ª porta → carry
);
```

⚠️ Frágil: reordenação nas portas do módulo original quebra silenciosamente todas as instâncias.

Por Nome — .porta(sinal) ← recomendado

```
somador_completo u1(
  .a(a_in),
  .c(c_in),
  .b(b_in),
  .carry(c_out),
  .soma(s_out)
);
```

✔️ Resistente a refatorações, auto-documentado e fácil de auditar em code-review.

Exemplo — Ripple Carry Adder 4 bits (4 somadores completos instanciados por nome)

```
module rca4(input [3:0] a, b, input cin, output [3:0] soma, output cout);
  wire [2:0] c;
  full_adder fa0(.a(a[0]),.b(b[0]),.cin(cin) ,.soma(soma[0]),.cout(c[0]));
  full_adder fa1(.a(a[1]),.b(b[1]),.cin(c[0]),.soma(soma[1]),.cout(c[1]));
  full_adder fa2(.a(a[2]),.b(b[2]),.cin(c[1]),.soma(soma[2]),.cout(c[2]));
  full_adder fa3(.a(a[3]),.b(b[3]),.cin(c[2]),.soma(soma[3]),.cout(cout));
endmodule
```

Verilog

FLUXO DE DADOS

Assign contínuo

```
module mux4x1_dataflow (
    input  wire [3:0] d,   // Entradas d0, d1, d2, d3
    input  wire [1:0] s,   // Seletor (s1 s0)
    output wire out
);
// Lógica: se s=3 seleciona d[3], senão se s=2 seleciona d[2]...
    assign out = (s == 2'b00) ? d[0] :
                 (s == 2'b01) ? d[1] :
                 (s == 2'b10) ? d[2] : d[3];
endmodule
```

Verilog

ESTRUTURAL + FLUXO DE DADOS

```
            // Módulo básico: Multiplexador 2x1
module mux2x1 (
    input wire a,     // Entrada quando sel = 0
    input wire b,     // Entrada quando sel = 1
    input wire sel,   // Pino de seleção
    output wire y     // Saída  (NÃO PODE SER REG em caso de uso de assign)
);
    assign y = sel ? b : a;
endmodule
            // Módulo Estrutural: Multiplexador 4x1
module mux4x1_structural (
    input wire [3:0] d,     // 4 entradas de dados
    input wire [1:0] s,     // 2 entradas de seleção
    output wire out         // Saída final
);
    wire w0, w1; // Fios internos para conectar os estágios
    // Estágio 1: Seleciona entre os pares de entrada usando o bit menos significativo (s[0])
    mux2x1 M0 (.a(d[0]), .b(d[1]), .sel(s[0]), .y(w0));
    mux2x1 M1 (.a(d[2]), .b(d[3]), .sel(s[0]), .y(w1));
    // Estágio 2: Seleciona entre os resultados do estágio 1 usando o bit mais significativo (s[1])
    mux2x1 M2 (.a(w0),   .b(w1),   .sel(s[1]), .y(out));
endmodule
```

Verilog

COMPORTAMENTAL
Bloco always

```
module mux4x1_behavioral (
    input  wire [3:0] d,   // Entradas de dados
    input  wire [1:0] s,   // Seletor de 2 bits
    output reg  out  // Saída (reg para o bloco always)
);

    // O bloco 'always' monitora mudanças em 'd' ou 's'
    always @(*) begin
        case (s)
            2'b00:   out = d[0];
            2'b01:   out = d[1];
            2'b10:   out = d[2];
            2'b11:   out = d[3];
            default: out = 1'bx; // Define como 'desconhecido' por segurança
        endcase
    end

endmodule
```

Verilog

COMPORTAMENTAL
Bloco always

```
module decod_behavioral (
    input  instr[7:0],   // Entradas de dados
    output reg  type  // Saída (reg para o bloco always)
);

    // O bloco 'always' monitora mudanças em 'instr'
    always @(*) begin
        casez (istr[7:5)
            3'b1??:   type = 2'd0;
            3'b01?:   type = 2'd1;
            3'b11?:   type = 2'd2;
            default:  type = 2'd3;
        endcase
    end

endmodule
```

case vs casex vs casez — tratamento de X e Z

Instrução	Valor 0	Valor 1	Valor X	Valor Z	Char ?
case	0	1	X	Z	X
casex	0	1	dc	dc	X
casez	0	1	X	dc	dc

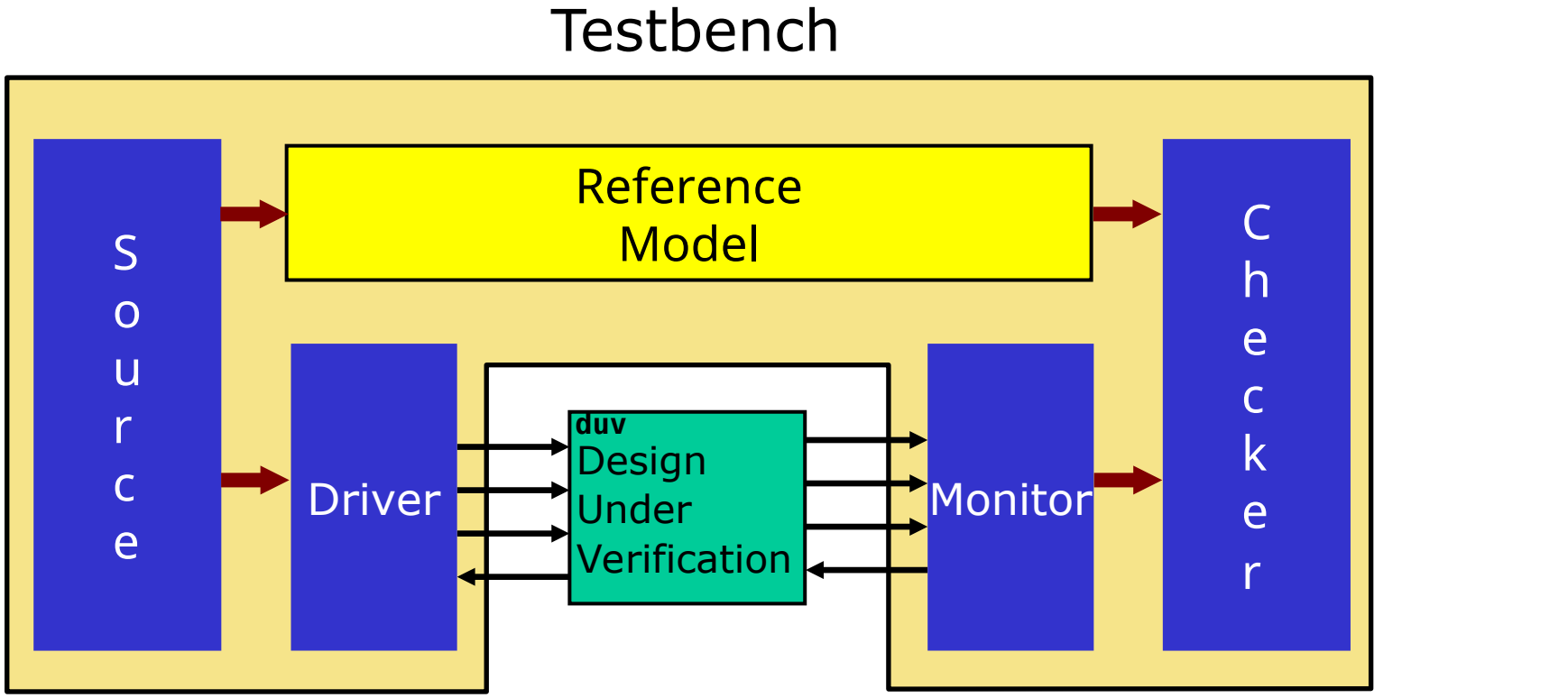
dc = don't care | casez é preferido ao casex em síntese moderna

Verilog

COMPORTAMENTAL
Bloco always

```
module mux4x1_ifelse (
    input  wire [3:0] d,   // Entradas d0 a d3
    input  wire [1:0] s,   // Seletor de 2 bits
    output reg  out  // Saída (reg para atribuição no always)
);
    // O @(*) faz o bloco rodar sempre que d ou s mudarem
    always @(*) begin
        if      (s == 2'b00) out = d[0];
        else if (s == 2'b01) out = d[1];
        else if (s == 2'b10) out = d[2];
        else                 out = d[3]; // Caso s == 2'b11
    end
endmodule
```

Verificação: testbench



Verificação: testbench

Principais Funções:

- **Geração de Estímulos**: Cria sinais de entrada (vetores de teste) para alimentar o dispositivo que está sendo testado (conhecido como DUV, UUT - Unit Under Test, DUT – Design Under Test, etc)
- **Aplicação de Sinais**: Gerencia o tempo e a sequência em que as entradas são alteradas para simular condições reais de operação
- **Verificação de Resultados**: Compara automaticamente as saídas geradas pelo circuito com os resultados esperados, muitas vezes utilizando comandos como assert para reportar erros
- **Monitoramento**: Registra o comportamento dos sinais internos e externos para análise posterior em formas de onda (gráficos)

Verificação: testbench

Características:

- **Entidade Vazia**: Ao contrário dos componentes de hardware reais, a entidade de um testbench não possui portas de entrada ou saída físicas, pois ele é um "sistema fechado" que contém tudo o que é necessário para o teste
- **Instanciação de Componentes**: Ele declara e "instancia" o código do circuito real (DUT) como um componente interno para interconectá-lo aos sinais de teste
- **Sinais Internos**: Utiliza sinais auxiliares (como wire) para conectar as "saídas" do gerador de estímulos às "entradas" do componente testado
- **Não Sintetizável**: O código de um testbench é feito apenas para simulação; ele contém comandos de tempo (como wait for 10ns) que não podem ser transformados em circuitos físicos reais

Verificação: testbench

TESTBENCH

```
module tb_somador;
    reg a, b, cin;
    wire s, cout;
    // Instancia o somador
    somador_completo uut (a, b, cin, s, cout);
    initial begin
       $dumpfile("dump.vcd");
       $dumpvars(1);
    end
    initial begin
     // Exibe resultados no console
     $monitor("A=%b B=%b Cin=%b -> S=%b Cout=%b", a, b, cin, s, cout);

     // Testa todas as combinações
     a=0; b=0; cin=0; #10;
     a=0; b=1; cin=0; #10;
     a=1; b=0; cin=0; #10;
     a=1; b=1; cin=0; #10;
     a=0; b=0; cin=1; #10;
     a=0; b=1; cin=1; #10;
     a=1; b=0; cin=1; #10;
     a=1; b=1; cin=1; #10;
     $finish;
    end
endmodule
```

DUT

```
module somador_completo (
    input a,
    input b,
    input cin,
    output s,
    output cout
);

 assign s = a ^ b ^ cin;
 assign cout = (a & b) | (cin & (a ^ b));
endmodule
```